

DevOps Report for Your-Docs

1. Project Overview

Your-Docs is deployed with a split architecture:

- Frontend hosted on Vercel
- Backend hosted on AWS EC2
- Backend services packaged as Docker containers
- Nginx used as a reverse proxy
- PostgreSQL used as the application database
- HTTPS enabled with Let's Encrypt
- Jenkins prepared as the CI/CD automation layer

This setup moves the backend away from Render free-tier cold-start behavior and gives more direct control over uptime, deployment, and infrastructure.

2. Why DevOps Was Needed in This Project

This project includes multiple moving parts:

- a React frontend
- several Python microservices
- authentication and document APIs
- websocket-based collaboration
- a prediction service
- a database

If these pieces are managed manually, deployment becomes slow and error-prone. DevOps practices were introduced so the project can:

- deploy consistently
- run the same way across environments
- keep secrets out of source control
- reduce manual setup on the server
- support repeatable deployment through Jenkins

3. Problems in the Earlier Deployment

The backend was previously deployed on Render free services. That created an operational issue:

- the service spun down after inactivity
- the first request in the morning hit a sleeping backend
- login requests returned `502 Bad Gateway`

This was not a frontend bug. It was an infrastructure behavior caused by the platform's free-tier sleep model.

4. New Deployment Architecture

The current deployment architecture is:

1. User opens the frontend on Vercel. 2. The frontend sends API requests to `https://yourdocs.webhop.me``. 3. Nginx on AWS EC2 receives the request. 4. Nginx forwards the request to the gateway container on port ``8000``. 5. The gateway routes the request to internal backend services. 6. Internal services read and write data using PostgreSQL.

This design separates responsibilities cleanly:

- Vercel serves static frontend files
- EC2 hosts backend runtime and supporting services
- Nginx handles public web traffic
- Docker Compose manages service orchestration

5. AWS EC2 in This Project

What EC2 Is

Amazon EC2 is a virtual server in the cloud. In this project, the EC2 instance acts as the backend host machine.

It runs:

- Ubuntu Linux
- Docker Engine
- Docker Compose
- Nginx
- the backend containers
- the PostgreSQL container

Why EC2 Was Chosen

EC2 was chosen because:

- it does not auto-sleep like Render free web services
- it provides direct SSH access
- Docker-based deployments work well on it
- it fits a low-cost or free-tier learning setup

Important EC2 Operational Behavior

- If the instance is **running**, the backend can be accessed and Jenkins can deploy to it.
- If the instance is **stopped**, the backend is unavailable and Jenkins cannot SSH into it.
- When an EC2 instance is stopped and started again, the public IP can change unless a fixed Elastic IP is used.

That means in the current setup:

- deployment requires the instance to be **running**
- if the IP changes, the free hostname may need to be updated

6. SSH Access and Server Authentication

Access to the EC2 server is done using SSH and a `.pem`` private key.

Example command used:

```
ssh -i backend_docs.pem ubuntu@EC2_PUBLIC_IP
```

Meaning:

- `ssh` starts a secure shell session
- `-i` specifies the private key file
- `ubuntu` is the default username for the Ubuntu EC2 image
- `EC2\_PUBLIC\_IP` is the public address of the server

This is the secure remote administration mechanism used for:

- first-time server setup
- checking logs
- running deployment commands manually
- allowing Jenkins to deploy over SSH

7. Docker Usage in the Project

Why Docker Is Used

Docker packages each backend service together with its runtime and dependencies. This avoids "works on my machine" problems and makes deployment reproducible.

Containers Used

The backend deployment includes these containers:

- `gateway-service`
- `auth-service`
- `docs-service`
- `websocket-service`
- `prediction-service`
- `postgres`

Benefits of Containerization Here

- services are isolated from each other
- deployments are repeatable
- server setup is lighter than installing each service manually
- rollback and restart operations are easier
- local and remote deployment patterns become similar

8. Docker Compose Usage

Docker Compose is used to run the full backend stack from one file:

- `backend/docker-compose.aws.yml`

It defines:

- which services exist

- which Dockerfiles build them
- which environment variables are passed in
- which ports are exposed
- service dependency order
- health checks
- the persistent volume for PostgreSQL

Important Compose Command

```
docker compose --env-file .env.aws -f docker-compose.aws.yml up -d --build
```

This command means:

- `--env-file .env.aws`: load runtime configuration
- `-f docker-compose.aws.yml`: use the AWS deployment stack
- `up`: start the services
- `-d`: run in background
- `--build`: rebuild images before starting

Useful Operational Commands

```
docker compose --env-file .env.aws -f docker-compose.aws.yml ps
docker compose --env-file .env.aws -f docker-compose.aws.yml logs postgres
docker compose --env-file .env.aws -f docker-compose.aws.yml down
```

These help with:

- checking service status
- debugging startup problems
- stopping the environment cleanly

9. Environment Management

The backend configuration is stored in:

- `.env.aws.example` as a template
- `.env.aws` as the real runtime file on the server

Why This Matters

Sensitive information should not be hardcoded or committed publicly. This includes:

- database credentials
- secret keys
- API keys
- host configuration

Important Variables

- `POSTGRES_DB`
- `POSTGRES_USER`
- `POSTGRES_PASSWORD`

- ``SQL_DATABASE_URL``
- ``SECRET_KEY``
- ``FRONTEND_URL``
- ``CORS_ORIGINS``
- ``PREDICTION_API_KEY``

This pattern is a standard DevOps practice because it keeps:

- code separate from secrets
- deployment values environment-specific
- credentials outside Git history

10. PostgreSQL Usage

PostgreSQL is the project's main relational database.

Why It Is Used

It stores:

- users
- document metadata
- collaboration state
- session data
- application records

How It Is Hosted Here

For the current free-friendly deployment, PostgreSQL runs in a Docker container on the same EC2 instance.

Benefits:

- simpler setup than RDS
- lower learning overhead
- no extra managed service required during the first deployment

Tradeoff:

- the database is tied to the EC2 machine
- this is acceptable for a student/personal deployment but less ideal than a managed database for larger production workloads

11. Nginx Usage

What Nginx Is

Nginx is a web server and reverse proxy. In this project it sits in front of the backend containers.

Why Nginx Is Used

Nginx is responsible for:

- accepting public traffic on port ``80`` and ``443``
- forwarding API traffic to the backend gateway
- forwarding websocket traffic correctly

- acting as the integration point for HTTPS certificates

Reverse Proxy Role

Instead of exposing the gateway container directly to the public internet, Nginx handles incoming requests and proxies them internally to:

- ``127.0.0.1:8000``

This improves:

- security
- flexibility
- SSL handling
- public URL stability

12. HTTPS and SSL

The backend is publicly available through:

- ``https://yourdocs.webhop.me``

SSL was added using:

- Let's Encrypt
- Certbot

Why HTTPS Matters

The frontend is hosted on Vercel over HTTPS, so the backend should also be served securely. HTTPS is important for:

- browser trust
- secure login traffic
- avoiding mixed-content issues
- production-style deployment

Result

The backend health endpoint became available securely at:

- ``https://yourdocs.webhop.me/health``

13. Free Hostname Usage

Because there was no purchased custom domain, a free hostname was used:

- ``yourdocs.webhop.me``

This provides:

- a stable public name
- easier access than a raw IP address
- compatibility with Nginx and SSL setup

The main limitation is that if the EC2 public IP changes after stop/start, the hostname mapping may need to be updated.

14. Frontend and Backend Integration

The frontend remains on Vercel, while the backend runs on EC2.

The frontend should point to:

```
REACT_APP_API_URL=https://yourdocs.webhop.me
REACT_APP_WS_URL=wss://yourdocs.webhop.me
```

This ensures:

- API requests use the secure backend URL
- websocket connections use `wss`
- the frontend and backend work together across different hosting platforms

15. Jenkins CI/CD Usage

Why Jenkins Is Included

Jenkins is used as the CI/CD automation system for the backend.

It helps automate:

- pulling the latest code from GitHub
- preparing environment files using secure credentials
- validating Docker Compose configuration
- building backend Docker images
- deploying the backend to AWS EC2
- running a post-deploy health check

Jenkinsfile Used

The main automation definition is:

- `backend/Jenkinsfile`

Pipeline Flow

The current Jenkins pipeline works like this:

1. Developer updates code locally. 2. Code is pushed to GitHub. 3. Jenkins checks out the latest repository state. 4. Jenkins creates `.env.aws` at build time from Jenkins credentials. 5. Jenkins validates the Compose file. 6. Jenkins builds the Docker images. 7. Jenkins connects to EC2 over SSH. 8. Jenkins syncs backend files to the server using `rsync`. 9. Jenkins copies `.env.aws` to the remote backend directory. 10. Jenkins runs Docker Compose on the EC2 server. 11. Jenkins checks the public `/health` endpoint.

What Jenkins Credentials Store

Credentials are expected for values such as:

- SQL database URL
- backend secret key
- prediction API key
- EC2 SSH key

This is good DevOps practice because secrets remain outside Git.

Important Limitation

The current Jenkins pipeline can deploy to EC2 **only if the EC2 instance is running**.

If the instance is stopped:

- SSH will fail
- file sync will fail
- remote Docker commands will fail
- deployment will fail

So the current process is:

1. start EC2 2. wait until it is reachable 3. run Jenkins deployment

16. What Was Achieved Technically

This DevOps work achieved the following:

- moved the backend away from Render free-tier cold starts
- created a more stable backend environment on EC2
- containerized backend services for repeatable deployment
- added PostgreSQL to the deployment stack
- added Nginx reverse proxying
- enabled HTTPS
- added a public hostname
- prepared Jenkins-based CI/CD for backend automation

17. Benefits of the Current Setup

Operational Benefits

- more control over infrastructure
- no forced Render sleep behavior
- consistent deployments
- easier service debugging through Docker logs
- reusable deployment flow

Learning Benefits

This project now demonstrates hands-on experience with:

- Linux server administration
- cloud deployment on AWS
- SSH-based access
- Docker and Docker Compose
- reverse proxying with Nginx
- HTTPS setup with Certbot
- environment and secret management
- Jenkins CI/CD pipeline design

18. Current Risks and Practical Limitations

The setup is solid for learning and demos, but still has practical limitations:

- EC2 may need manual start before demos if it is intentionally stopped
- the public IP may change after restart
- the free hostname may need updating if IP changes
- the database is on the same machine as the app
- free-tier usage on AWS must be monitored
- exposed API keys should be rotated immediately if leaked

19. Recommended Future Improvements

If the project grows, the next DevOps upgrades could include:

- use a fixed Elastic IP or paid domain
- move PostgreSQL to a managed database such as AWS RDS
- add automatic EC2 start/stop scripting for Jenkins
- add image publishing to a registry
- add backup strategy for the database
- add monitoring and alerting
- add zero-downtime deployment strategy

20. Conclusion

DevOps in this project was not just about deployment. It was used to solve a real reliability issue, standardize the backend runtime, secure public traffic, and prepare the project for repeatable CI/CD.

The final system now combines:

- Vercel for frontend hosting
- AWS EC2 for backend hosting
- Docker for service packaging
- Docker Compose for orchestration
- PostgreSQL for persistent data
- Nginx for reverse proxying
- Certbot and Let's Encrypt for HTTPS
- Jenkins for automated deployment

This gives the project a practical production-style backbone while still staying approachable for learning and portfolio use.